
CS3460: Data Structures

Lab 06: Recursion

Total Points: 100

Problems

The $3n + 1$ Problem (60 points) **2**

The n -Queens Problem (40 points) **3**

Notes on Grading: Grading will be handled via Gradescope. Any file you submit must begin with the following package statement to be accepted:

```
1 package student;
```

Your code will be compiled and run against a series of tests. You may submit as many times as you would like before the assignment closes. To earn full credit, you must submit to both Gradescope and AsULearn before the assignment deadline.

Submission: Please submit the files `Collatz.java` and `NQueens.java`.

1. **The $3n + 1$ Problem (60 points):** Consider the following algorithm:

- (a) Start with any positive integer n .
- (b) If n is 1, halt.
- (c) If n is even, $n = n/2$, goto (b).
- (d) If n is odd, $n = 3n + 1$, goto (b).

In 1937, a German mathematician named Lothar Collatz conjectured that the algorithm always stops for any positive number n . Although this is not proven, it has been demonstrated to be true for all integers up to 100,000,000.

For any n , the sequence produced by this process is called the Collatz sequence, and the length of that sequence is called the Collatz length. For example, for $n = 53$, the Collatz length is 12, and the Collatz sequence is

$$53 \Rightarrow 160 \Rightarrow 80 \Rightarrow 40 \Rightarrow 20 \Rightarrow 10 \Rightarrow 5 \Rightarrow 16 \Rightarrow 8 \Rightarrow 4 \Rightarrow 2 \Rightarrow 1$$

Write a Java program called `Collatz.java` that accepts two positive integers a and b ($1 \leq a \leq b \leq 100000000$) and prints the number x ($a \leq x \leq b$) with the maximum Collatz length.

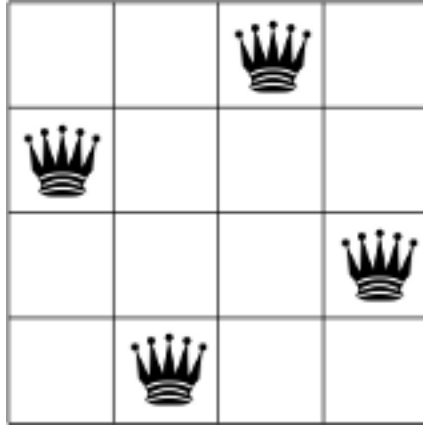
Your program should provide the following methods

- `public static int collatzLength(long x)` - returns the Collatz Length for x .
- `public static long longestInRange(long a, long b)` - returns the x ($a \leq x \leq b$) with the longest Collatz length.

Consider using memoization to reduce the number of redundant calculations you will need. You should also implement a main method to drive the program following the sample input/output shown below:

```
Enter the range: 1 1000000
The number with the greatest Collatz length in this range: 837799
The Collatz length of 837799 is 525
```

2. **The n -Queens Problem (40 points):** This problem asks you to place n queens on an $n \times n$ chess board such that none of the queens are capable of attacking another. A queen can move along any rank, file, or diagonal to see any square along any of those lines. The figure shows one possible solution for the 4-queens problem.



One possible solution for $n = 4$.

There might be many solutions to the n -queens problem. For $n = 8$, for example, there are 92 distinct solutions. Your task is to efficiently compute the number of valid solutions for any small value of n .

To solve this problem, you should generate the solutions recursively. Since queens attack along each rank (row), there can only be one queen per rank. This means that you can assign a queen to a location on each row and construct it row-by-row. For each row, try placing the queen at each possible location. If this creates a valid state (none of the existing queens are capable of attacking or being attacked by the newly added queen), then proceed to the next row, then when you return, remove the queen from that position and try another. If the state is not valid, there is no reason to recurse further (additional queens will not undo the violation).

Write a program `NQueens.java` that contains a method `public static int solve(int n)` that returns the number of valid configurations for n -queens (this will likely require to call a `private` recursive version of the method.) You should also implement a main method to drive the program following the sample input/output shown below:

```
Enter the number of queens: 8
The number of valid arrangements is 92
```